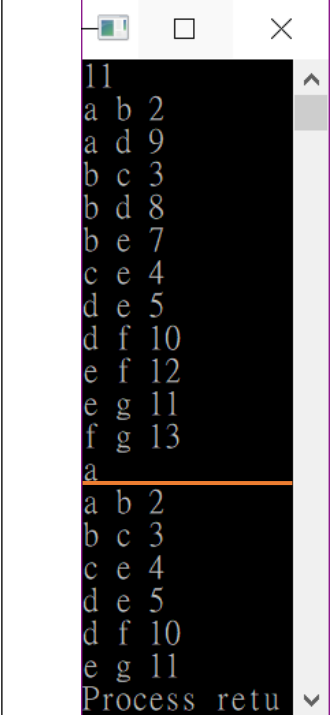
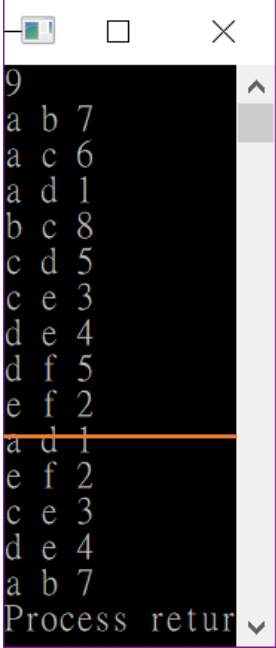
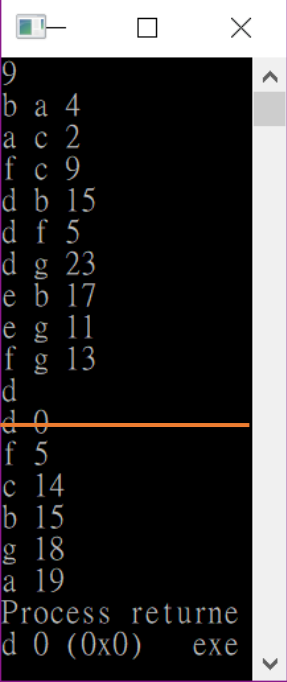


Shortest Path

一、執行畫面

Prim's Algorithm	Kruskal's Algorithm	Dijkstra's Algorithm
 <pre> 11 a b 2 a d 9 b c 3 b d 8 b e 7 c e 4 d e 5 d f 10 e f 12 e g 11 f g 13 a a b 2 b c 3 c e 4 d e 5 d f 10 e g 11 Process return </pre>	 <pre> 9 a b 7 a c 6 a d 1 b c 8 c d 5 c e 3 d e 4 d f 5 e f 2 a d 1 e f 2 c e 3 d e 4 a b 7 Process return </pre>	 <pre> 9 b a 4 a c 2 f c 9 d b 15 d f 5 d g 23 e b 17 e g 11 f g 13 d 0 f 5 c 14 b 15 g 18 a 19 Process return d 0 (0x0) exe </pre>

二、程式介紹

1. Prim's Algorithm

(1) 資料結構:

本程式中的 linked list 節點包含兩個端點、權重及指向下一個節點的 pointer。

```

typedef struct node{
    char v1;
    char v2;
    int weight;
    node* next;
} Node;

```

(2) 程式介紹:

本程式利用三個 linked list 及一個陣列「start」逐步跑出 minimum spanning tree。一開始程式將測資存入「graph」中，並依據使用者輸入的起點，選出候選的 edge，存入「fringe」，再從「fringe」中挑出權重最小的 edge 加入「tree」中。已經存在於 tree 中的端點會被加入「start」陣列中，並依據 start 中已包含的端點，刪掉「graph」中不能使用的 edge。

● void main()

在主函式中，使用者輸入 edge 與起點的資料後，會呼叫 prim 函式計

算出 minimum spanning tree, 並用 print 函式印出「tree」。

- void prim(Node** graph, char** start, Node** tree)
此函式會依據「start」陣列, 將「graph」linked list 中包含陣列中端點的 edge 加入暫存的「fringe」linked list 中, 並找出「fringe」中權重最小的 edge, 加入「tree」lined list; 同時將新的端點加入「start」陣列中。最後繼續呼叫 prim 函式, 直到沒有東西可以加入「fringe」linked list 中。
- void insert(Node** list, char v1, char v2, int weight)
此函式會將 edge 依據權重放入 linked list 中。
- void clean(Node** list);
此函式會將 linked list 全部清除, 歸還記憶體。
- void print(Node* list);
此函式用來印出整個 linked list。
- void del(Node** list, char* visited);
此函式會刪除「list」中左右端點都存在於「visited」陣列中的節點。

2. Kruskal's Algorithm

(1) 資料結構:

- 本程式的 linked list 節點包含兩個端點、權重及指向下一個節點的 pointer。

```
typedef struct node{
    char v1;
    char v2;
    int weight;
    node* next;
} Node;
```

- char forest[26]
此陣列表示 graph 中存在哪些端點, 且哪些端點在同一個 tree 中。例如 forest[0]、forest[3]、forest[25] 均為 0 時, 表示 a、d、z 節點已經在同一個 tree 中; 值小於零表示不存在該端點。端點一開始被放入此陣列中時, 其值為字元的十進位 (例如 graph 中存在端點 e 的話, forest[4]=101)。

(2) 程式介紹:

本程式將測資先存入「queue」linked list 中, 並將存在的端點存入「forest」陣列。接著找出「queue」中權重最小的 edge 加入「mst」linked list 中, 並將「forest」中兩個端點的值改為較小端點的值。直到所有的端點都被放入同一個 tree 中。

- void main()
主函式讓使用者輸入 edge 到「queue」linked list 中, 並將存在的端點存入「forest」陣列。接著呼叫 kruskal 函式, 並印出「mst」linked list。
- void kruskal(Node** queue, Node** mst, char* forest)
此函式會判斷目前「queue」中權重最小的 edge 的端點是否已經在同

一個 tree 中，不同的話便將 edge 加入「mst」，並將「forest」中的兩個端點改為同樣的值。接著將「queue」中已讀取過的 edge 刪除，然後重新呼叫 kruskal 函式，直到「queue」為空。

- void insert(Node** list, char v1, char v2, int weight)
此函式會將 edge 依據權重放入 linked list 中。
- void clean(Node** list)
此函式會將 linked list 全部清除，歸還記憶體。
- void print(Node* list)
此函式用來印出整個 linked list。

3. Dijkstra's Algorithm

(1) 資料結構:

- int graph[i][j]
本程式依據使用者輸入的 edge 數量，分配不同大小的二維陣列，藉以存放 graph。其表示由 i 到 j 的 edge 的權重。如果 graph 中不存在該端點，則值小於零。
- int result[i]
此陣列表示起點到不同端點的 cost。例如 result[3] 為 8 的話，表示起點到 d 的 cost 為 8。

(2) 程式介紹:

- void main()
主函式將測資存入 graph 中、將「result」中的起點權重設為 0，接著呼叫 dijkstra 函式，最後將「result」依據權重大小印出。
- void dijkstra(int** graph, int amount, char source, int** result)
此函式會一一判斷起點到某一點是否可以走，接著判斷是新的路的權重還是「result」中已存的權重比較小。接著呼叫 dijkstra 函式繼續判斷新的路是否可以繼續走。

三、心得

對應不同的演算法，存取資料的方式很重要。用了不好的方式存資料，在處理資料的時候，可能就會變得很複雜，可讀性很差，也容易一直寫錯。我本來想要沿用前兩個演算法的函式在 Dijkstra's Algorithm 上，結果發現用陣列的方式簡單很多，所以乾脆全部重寫。或許 Prim's Algorithm 可以用別的方式寫，我想我還可以再多試一下不同的寫法，讓程式碼變得直觀一些，自己也比較好懂；但 Kruskal's Algorithm 本來就是以 edge 為主的演算法，可能維持現在的方法是比較方便的。